# Width Speeds Up Memorisation Faster Than Generalisation: An Empirical Study of Delayed Generalisation on Modular Addition

**Anonymous (autonomous-agent research run)**

## Abstract

Delayed generalisation ("grokking") on modular arithmetic is mostly studied at one model size, so how it depends on scale is poorly characterised. We state a falsifiable "two-clock" account—a memorisation clock that speeds up with width and a generalisation clock governed by the weight-decay time scale—and register seven predictions before running the main sweeps. We then test them with 434 full-batch AdamW runs (6 seeds per width in the core sweep; 2–6 elsewhere) on modular addition with MLPs (24–512 hidden units), plus other operations, a modulus change, width-dependent learning-rate scalings and a small transformer. In the core setting ($p$=53, 30% of pairs for training), time to fit the training set falls as $w^{-1.0}$ to $w^{-1.4}$, whereas time to generalise falls more slowly (log-log slope $-0.56$, 95% CI $[-0.77, -0.36]$ for $w \geq 128$), so the *relative* delay $t_{val}/t_{train}$ grows with width while the *absolute* delay shrinks. The growth of the relative delay is not robust: it vanishes under a $\mathrm{lr} \propto 1/w$ parameterisation that keeps the decay rate $\mathrm{lr} \cdot \mathrm{wd}$ fixed, reverses (wider models generalise later) when the decay rate falls with width, and is absent for $p$=97 and for some operations. Memorisation time is indistinguishable between true and random labels (Spearman $\rho$=0.9999 over 28 width–data-fraction cells), and fitting requires only ≈4–5 parameters per training example, but generalising needs several times more width: narrow models that fit the training set stay at chance for $10^5$ steps and generalise only after $2\text{–}3 \times 10^5$ steps, so the "threshold" is a budget-dependent soft boundary. Of the seven registered predictions, three are supported, three are partly supported (one component falsified in each) and one is falsified: at fixed width, generalisation time does not depend on learning rate and weight decay only through their product; a post-hoc test shows that a product-like dependence emerges only at large width. Scope limits: one small modulus, one optimiser recipe, $\leq 6$ seeds, and the transformer evidence is weak.

## 1 Introduction

Neural networks trained on small algorithmic datasets can reach perfect training accuracy long before validation accuracy rises [Power et al., 2022]. Explanations include mechanistic circuit formation [Nanda et al., 2023], a loss-landscape/weight-norm picture [Liu et al., 2023], efficiency of competing circuits under weight decay [Varma et al., 2023], and a lazy-to-rich transition of the training dynamics [Kumar et al., 2024]. Most such studies fix the architecture size. Yet experimenters routinely choose the width of the model when building grokking testbeds, and the choice can change both *whether* and *how strongly* delayed generalisation appears. We ask: *on modular arithmetic, how does the occurrence of delayed generalisation depend on model scale?*

Preprint.

Rather than just plotting delay against size, we put forward an explanation that makes checkable predictions, state the predictions before the main experiments, and report which are falsified. The explanation (Sec. 3.1) says that two "clocks" scale differently with width: fitting the training labels (memorisation) speeds up quickly with width, while generalisation is released on a time scale set by weight decay. Our contributions are empirical:

- A measurement protocol with bootstrap uncertainty and explicit handling of censored runs (as lower bounds), and a self-registered predictions file written before the main sweeps (`code/PREREG.md`).
- In the core setting, the memorisation clock scales as $w^{-1.0}\dots w^{-1.4}$ and is label-agnostic, whereas the generalisation clock scales much more weakly at large width (steeply near a threshold), so relative delay grows and absolute delay shrinks with width (Sec. 4.1,4.2).
- Fitting and generalising have very different capacity thresholds, and the latter is a soft, budget-dependent boundary; with more data, narrow models generalise without any delay (Sec. 4.3).
- The width–delay relation is *parameterisation dependent*: depending on how lr and wd scale with width, relative delay grows, stays flat, or grows faster, and absolute generalisation time can even increase with width (Sec. 4.4,4.5).
- An honest scorecard (Table 3): of seven registered predictions, 3 are supported (P1, P2 in the core setting, P7), 3 are partly supported (P3, P5, P6) and 1 is falsified (P4).

## 2 Related work

**Grokking.** Power et al. [2022] introduced delayed generalisation on algorithmic datasets, including modular arithmetic. Nanda et al. [2023] reverse-engineer a one-layer transformer on modular addition and define progress measures; Liu et al. [2023] relate grokking to weight norm and initialisation scale and extend it beyond algorithmic data; Varma et al. [2023] explain it via the relative efficiency of memorising and generalising circuits and test predictions by training experiments; Kumar et al. [2024] connect it to a lazy-to-rich dynamics transition in two-layer networks. Davies et al. [2023] discuss grokking together with double descent [Nakkiran et al., 2020]. We do not test any of these mechanisms directly (we do no interpretability); we use them to motivate predictions on how timing should scale with width, learning rate and weight decay.
**Memorisation capacity.** Zhang et al. [2017] show that networks fit random labels; Tirumala et al. [2022] and Morris et al. [2025] study memorisation in language models, the latter with a method that estimates how much information a model holds about its data points. Our random-label probe is a small-scale analogue used to time the "memorisation clock".
**Optimiser.** We use AdamW [Loshchilov and Hutter, 2017], in which weight decay shrinks weights by a factor $(1-\mathrm{lr}\cdot\mathrm{wd})$ per step, which is why the product $\mathrm{lr}\cdot\mathrm{wd}$ appears in our time-scale prediction.
**Positioning.** Prior work (e.g. Power et al., 2022, Liu et al., 2023) varies data fraction, weight decay and initialisation scale; here we focus on *model width as the controlled variable*, separate absolute from relative delay, and stress-test conclusions against optimiser parameterisation and censoring.

## 3 Setup, hypothesis and registered predictions

**Data.** For a prime $p$ and operation $\circ$, all $p^2$ pairs $(a, b)$ are enumerated with label $a \circ b \bmod p$ (add, sub, mul, or $a^2+b^2$); a random fraction $f$ (default $0.3$) is the training set and the rest the validation set. Main modulus: $p$=53 (843 training pairs); $p$=97 is a robustness check. Data are generated in memory (no download).
**Models.** (i) MLP: learned $w$-dim embeddings of $a$ and $b$, concatenated, two hidden ReLU layers of width $w$, linear readout to $p$ logits (4k–842k parameters for $w$=24–512). (ii) One-layer transformer ($d_{model}$=$w$, 4 heads, MLP $4w$, no LayerNorm; input tokens $[a, b, =]$). Initialisation: all weight matrices $\mathcal{N}(0, 1/\text{fan-in})$ (He scale for MLP hidden layers), i.e. a standard ("SP") parameterisation.
**Training.** Full-batch AdamW ($\beta$=$(0.9, 0.98)$, lr $10^{-3}$, wd 1.0, 10-step warm-up), cross-entropy, up to $T_{max}$ steps ($10^5$ for most MLP runs). Accuracy is evaluated every 10 steps up to step 500, every 50 up to 5000, then every 200. Each *seed* controls the initialisation and the train/validation split.
**Metrics.** $t_{\mathrm{train}}$ ($t_{\mathrm{val}}$): first evaluation step with train (validation) accuracy $\geq 0.99$. Relative delay $R = t_{\mathrm{val}}/t_{\mathrm{train}}$, absolute delay $D = t_{\mathrm{val}} - t_{\mathrm{train}}$. Runs that do not reach the threshold by $T_{max}$ are

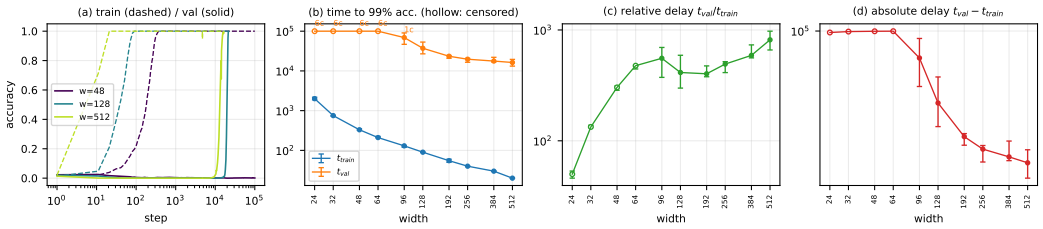



Figure 1: Core MLP width sweep (modular addition, $p$=53, $f$=0.3, 6 seeds/width). (a) Example runs (seed 100). (b) Time to 99% accuracy; median and bootstrap 95% CI; hollow markers: $\geq$ half the runs censored at $10^5$ steps. (c) Relative delay $R$. (d) Absolute delay $D$.

*censored* and their value is set to $T_{max}$ (a lower bound); medians over seeds are therefore bounds when $\geq$ half of the runs are censored (hollow markers, "$\geq$" in tables). Uncertainty: 95% bootstrap CI of the median (4000 resamples of seeds); log-log slopes are fit to per-width medians, with CIs from bootstrap resampling of seeds within width. $t_{\text{train}}$ has a resolution of 10 steps for $t_{\text{train}} \leq 500$ (e.g. $\pm 25\%$ at $t_{\text{train}}$=20), which bootstrap CIs do not capture. Slopes and rank correlations use only widths where fewer than half of the runs are censored unless stated.

### 3.1 Explanation under test: a "two-clock" account

*Memorisation clock* $t_{mem}(w)$: how fast the network fits training labels; it is limited by memorisation speed and should shorten quickly with width. *Generalisation clock* $t_{gen}$: when weight decay has pruned the memorising solution enough for the generalising circuit to take over; it should be set mainly by the optimiser time scale $1/(\text{lr} \cdot \text{wd})$ and depend only weakly on width. Registered predictions (full text with falsification criteria in `code/PREREG.md`; written after an exploratory pilot, seeds 0–2, but before the sweeps with seeds 100+):

**P1** $t_{\text{train}}$ decreases monotonically with width (negative log-log slope).

**P2** $R$ increases with width while $D$ does not increase.

**P3** Random-label fitting time tracks real-label $t_{\text{train}}$ (Spearman $> 0.8$), and the smallest width that fits random labels predicts the smallest width that fits true labels within a factor 1.5, shifting to larger widths with more training data.

**P4** At fixed width, $t_{\text{val}}$ depends on lr and wd through lr·wd: log-log slope in $[-1.3, -0.7]$, and equal-product pairs differ by $< 1.5\times$.

**P5** P1 and P2 hold for sub, mul, $a^2+b^2$ and $p$=97.

**P6** P1, P2 hold for the transformer.

**P7** (exploratory) The width trend changes if lr is scaled with width, i.e. it depends on the parameterisation.

A post-hoc, clearly exploratory prediction P8 (Sec. 4.4) was registered after the main sweeps and before its test.

## 4 Experiments and results

All experiments use the code in `code/`; Appendix A lists which script and result file produce each figure and table. Counts of runs per cell are given in tables; because $n \leq 6$ seeds per cell, CIs are coarse and we avoid claims resting on small differences. No multiple-comparison correction was applied: we report effect sizes with CIs rather than p-values, and no conclusion rests on a single borderline test.

### 4.1 Core sweep: scale makes the delay relatively longer but absolutely shorter

Figure 1 and Table 1 show the core sweep. Training accuracy reaches 99% in $t_{\text{train}} = 2000$ steps at $w$=24 and 20 steps at $w$=512. Over the ten widths the log-log slope of $t_{\text{train}}$ is $-1.39$ $[-1.40, -1.37]$; restricted to $w \geq 128$ it is $-1.04$ $[-1.06, -1.02]$ (Spearman $\rho = -0.997$ over 30

Table 1: Core sweep. Median [95% bootstrap CI] over 6 seeds; steps. cens.: runs not reaching 99% validation accuracy within $10^5$ steps. For widths $\leq 64$, $t_{val}$, $R$, $D$ are lower bounds (censored).

| width | params | $t_{train}$ | $t_{val}$ | $R$ | $D$ | cens. | $n$ |
|---|---|---|---|---|---|---|---|
| 24 | 4k | 2000 [1875,2175] | $\geq$100000 [100000,100000] | $\geq$50 [46,53] | $\geq$98000 [97825,98125] | 6/6 | 6 |
| 32 | 7k | 750 [725,750] | $\geq$100000 [100000,100000] | $\geq$133 [133,138] | $\geq$99250 [99250,99275] | 6/6 | 6 |
| 48 | 12k | 330 [320,350] | $\geq$100000 [100000,100000] | $\geq$303 [286,312] | $\geq$99670 [99650,99680] | 6/6 | 6 |
| 64 | 19k | 210 [210,225] | $\geq$100000 [100000,100000] | $\geq$476 [445,476] | $\geq$99790 [99775,99790] | 6/6 | 6 |
| 96 | 38k | 130 [120,130] | 69100 [46700,90400] | 555 [373,695] | 68975 [46575,90270] | 1/6 | 6 |
| 128 | 63k | 90 [90,90] | 37200 [26900,53200] | 413 [299,591] | 37110 [26810,53110] | 0/6 | 6 |
| 192 | 131k | 55 [50,60] | 23400 [20800,24400] | 402 [378,474] | 23345 [20745,24345] | 0/6 | 6 |
| 256 | 224k | 40 [40,40] | 19700 [16500,20700] | 492 [412,518] | 19660 [16460,20660] | 0/6 | 6 |
| 384 | 484k | 30 [30,30] | 17700 [16800,22000] | 590 [560,733] | 17670 [16770,21970] | 0/6 | 6 |
| 512 | 842k | 20 [20,20] | 16300 [13200,19500] | 815 [660,975] | 16280 [13180,19480] | 0/6 | 6 |

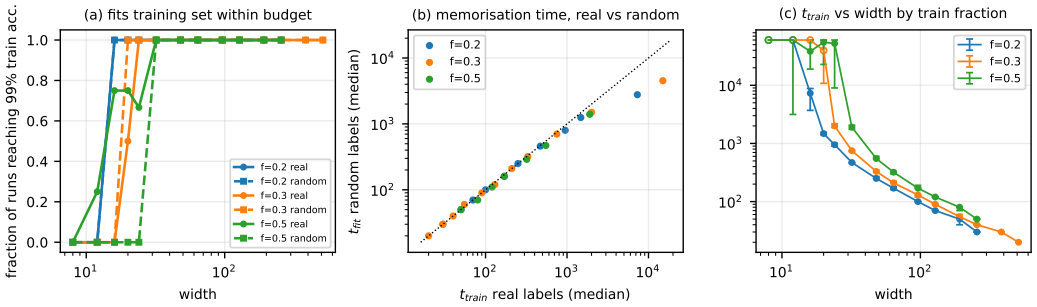



Figure 2: Memorisation probe (MLP, $p$=53, three training fractions $f$; random-label runs use the same training inputs with uniformly random labels). (a) Fraction of runs reaching 99% train accuracy within budget (60–100k steps; 3–6 seeds). (b) Median fitting time, random vs. real labels, one point per (width, $f$) cell where both fit. (c) $t_{\text{train}}$ vs width, real labels (hollow: censored).

runs): **P1 is supported**. Validation accuracy tells a different story. For $w \leq 64$ none of 24 runs reaches 99% validation accuracy in $10^5$ steps (final validation accuracy $\leq 0.07$, i.e. near chance $1/53$), at $w$=96 five of six runs do, and for $w \geq 128$ all do. Among $w \geq 128$, $t_{\text{val}}$ falls from 37.2k to 16.3k steps: slope $-0.56$ $[-0.77, -0.36]$ (Spearman $\rho = -0.81$ $[-0.92, -0.60]$), i.e. shallower than $t_{\text{train}}$ (slope difference $\approx 0.5$). Consequently $R$ rises from 413 ($w$=128) to 815 ($w$=512) (slope $+0.49$ $[+0.25, +0.71]$; $\rho = +0.78$ $[0.51, 0.94]$), while $D$ falls from 37k to 16k (slope $-0.56$ $[-0.77, -0.34]$): **P2 is supported for this setting**. $R$ is not monotone in $w$ at intermediate widths ($555, 413, 402, 492, 590, 815$ for $w$=$96 \ldots 512$), so the claim is about the trend. The local slope of $t_{\text{val}}$ is much steeper at small width ($-2.2$ for $96 \to 128$, $-1.1$ for $128 \to 192$, $-0.6$ for $192 \to 256$) than at large width ($-0.26$ for $256 \to 384$, $-0.29$ for $384 \to 512$): $t_{\text{val}}$ appears to flatten towards a floor of roughly $1.5 \times 10^4$ steps, about $15/(\text{lr} \cdot \text{wd})$. The final parameter $L_2$ norm is 41–48 for all widths $\geq 24$, whether the model memorised or generalised, so this norm does not distinguish the two phases here.

### 4.2 The memorisation clock is label-agnostic

We repeat training on random labels (same inputs, 3 seeds per cell, 28 (width, $f$) cells where both label types fit). Median fitting times for random and true labels are almost identical (Fig. 2b; Spearman $\rho = 0.9999$ $[0.998, 1.0]$ on log scale, median ratio 0.96): the time to fit is a property of the model and training-set size rather than of label structure, supporting the interpretation of $t_{\text{train}}$ as a memorisation clock (first half of **P3 supported**). With random labels the smallest width at which at least half of runs fit is 16, 20, 32 for $f$=$0.2, 0.3, 0.5$, i.e. 2.5k, 3.4k, 6.5k parameters for 562, 843, 1404 training pairs: $\approx 4.4$, 4.0, 4.6 parameters per training example (wd= 1, budget $\leq 10^5$ steps). So capacity needed for *fitting* grows roughly proportionally to the amount of data. The second half of P3 is **falsified at** $f$=0.5: true labels are fitted from $w$=16 (3/4 runs) while random labels need $w$=32 (a factor $2 > 1.5$), because small models can fit true labels by learning the rule (Sec. 4.3). At $f$=0.2 and 0.3 the thresholds agree (16 vs 16; 20 vs 20), and thresholds shift to larger width with more data as predicted (random labels).

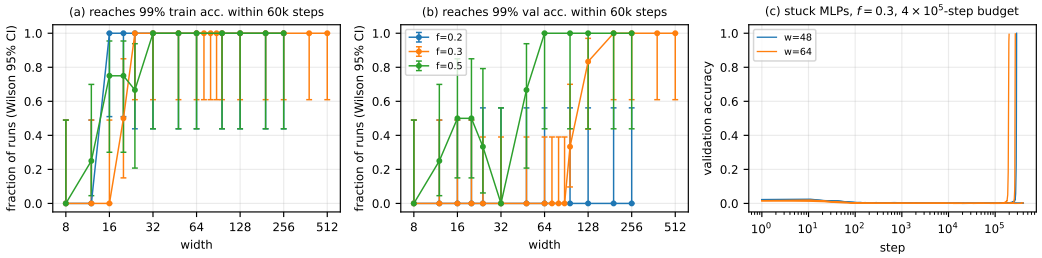



Figure 3: (a,b) Fraction of runs reaching 99% train / validation accuracy within 60k steps vs width, for three training fractions (3–6 seeds per cell; Wilson 95% CIs). (c) Validation accuracy of narrow MLPs ($f$=0.3) that stayed at chance in the $10^5$-step runs, with a $4 \times 10^5$-step budget (3 seeds each).

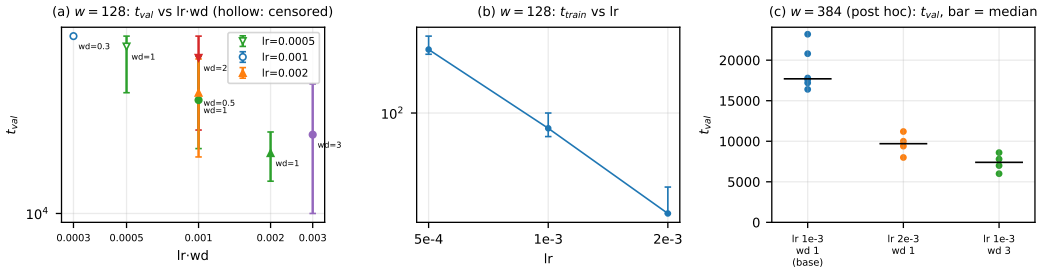



Figure 4: (a,b) MLP $w$=128, 7 (lr, wd) settings, 4 seeds each, $T_{max} = 8 \times 10^4$: $t_{\text{val}}$ vs lr·wd (median, bootstrap CI; marker = lr, colour = wd; (lr=$10^{-3}$, wd= 0.3) censored in all runs) and $t_{\text{train}}$ vs lr. (c) $w$=384, post-hoc test P8 (individual runs).

### 4.3 Fitting is cheap; generalising needs far more width, and the boundary is soft

At $f$=0.3 the smallest fitting width is $\approx 20$ but models with $w$ up to 88 remain at chance validation accuracy for $10^5$ steps (generalisation by $10^5$ steps: $w$=72: 1/6, 80: 1/6, 88: 4/6, 96: 5/6, 128: 6/6). The boundary is a budget effect, not a hard capacity limit: with a $4 \times 10^5$-step budget, 2/3 runs at $w$=64 ($t_{val}$=1.99 and $2.70 \times 10^5$) and 1/3 at $w$=48 ($2.88 \times 10^5$) do generalise (Fig. 3c; the other runs are still at chance). With $f$=0.2 no run generalises within 60k steps up to $w$=256 (3 seeds each; budget-limited, not shown to be impossible). With $f$=0.5 the generalising width is much smaller (3/3 runs from $w$=64 within 60k steps) and very narrow models ($w$=12–24) sometimes generalise *without delay*: 6 runs have $R \leq 3.2$ (all at $f$=0.5, $w \in \{12, 16, 20, 24\}$; 5 with $R \leq 1.35$). They are too small to memorise 1404 pairs and must learn the rule directly; in absolute terms they are slow ($t_{\text{val}}$ up to $6 \times 10^4$ steps). So the picture is three-regimes-in-width for a given data fraction: too small to fit; fits only by memorisation (generalisation slow or absent within the budget); and a regime where generalisation arrives after a delay that shrinks in absolute terms with width. Delay as such exists only in the middle and upper regimes. We did not map the full $(f, w)$ phase diagram.

### 4.4 Optimiser time scale

P4 predicts that $t_{\text{val}}$ depends on lr and wd only via lr·wd. At $w$=128 the log-log slope of $t_{\text{val}}$ on lr·wd is $-0.62$ $[-0.83, -0.42]$ (run-level bootstrap; 7/28 runs censored, which biases the slope towards 0), outside the registered $[-1.3, -0.7]$. Three settings with identical product $10^{-3}$ (lr $5\times10^{-4}, 10^{-3}, 2\times10^{-3}$) give median $t_{\text{val}}$ of 62k (1/4 censored), 37.8k and 41.2k: ratio $1.64 > 1.5$. A regression of $\log t_{\text{val}}$ on $\log$ lr and $\log$ wd gives exponents $-0.82$ $[-1.09, -0.54]$ for lr and $-0.55$ $[-0.78, -0.33]$ for wd. **P4 is falsified** as registered at $w$=128. The memorisation clock is mostly an lr clock: $t_{\text{train}} \propto \text{lr}^{-0.77}$.

*Post-hoc saturation test (P8).* Since $t_{\text{val}}$ flattens at large width, one expects it to be controlled by the optimiser at large width. At $w$=384 (4 seeds per setting; baseline 6 seeds), doubling lr reduces median $t_{\text{val}}$ from 17.7k to 9.7k (ratio 1.82 [1.58, 2.39]) and tripling wd to 7.4k (ratio 2.39 [2.06, 3.2]): both

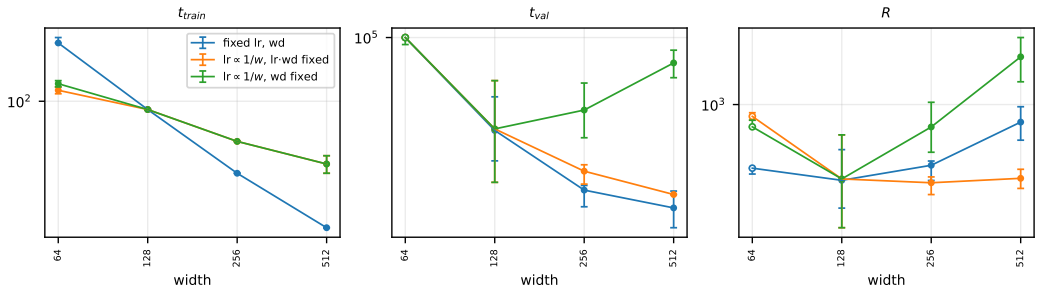



Figure 5: Width dependence under three scalings of lr/wd with width (MLP, $p$=53; reference $w$=128, lr $10^{-3}$, wd 1). Fixed lr, wd (6 seeds); lr$\propto 1/w$ with lr·wd fixed (4 seeds; 2 at $w$=512); lr$\propto 1/w$ with wd fixed (same seeds). Hollow markers: censored.

Table 2: Other operations and modulus (MLP, $f$=0.3, lr $10^{-3}$, wd 1). Median [95% CI]; $\geq$: at least half of the runs censored at $10^5$ steps.

| task | $p$ | width | $t_{train}$ | $t_{val}$ | $R = t_{val}/t_{train}$ | cens./$n$ |
|---|---|---|---|---|---|---|
| add | 53 | 64 | 210 | $\geq$100000 [100000,100000] | $\geq$476 | 6/6 |
| add | 53 | 128 | 90 | 37200 [26900,53200] | 413 | 0/6 |
| add | 53 | 256 | 40 | 19700 [16500,20700] | 492 | 0/6 |
| add | 97 | 64 | 470 | $\geq$100000 [100000,100000] | $\geq$213 | 3/3 |
| add | 97 | 128 | 160 | 54800 [51400,58200] | 342 | 0/3 |
| add | 97 | 256 | 70 | 13600 [12800,14400] | 194 | 0/3 |
| mul | 53 | 64 | 210 | $\geq$100000 [100000,100000] | $\geq$476 | 4/4 |
| mul | 53 | 128 | 90 | 39000 [28400,45800] | 433 | 0/4 |
| mul | 53 | 256 | 40 | 16500 [15600,18400] | 412 | 0/4 |
| sq | 53 | 64 | 200 | $\geq$83700 [66600,100000] | $\geq$407 | 2/4 |
| sq | 53 | 128 | 80 | $\geq$78300 [24200,100000] | $\geq$909 | 2/4 |
| sq | 53 | 256 | 40 | 25100 [17800,33200] | 628 | 0/4 |
| sub | 53 | 64 | 210 | $\geq$100000 [100000,100000] | $\geq$476 | 4/4 |
| sub | 53 | 128 | 90 | 30800 [29400,35800] | 342 | 0/4 |
| sub | 53 | 256 | 40 | 22000 [15600,24000] | 550 | 0/4 |

exceed the registered factor 1.4 (**P8 supported**). Together with P4: the generalisation clock follows lr·wd approximately (exponents $\approx -0.8$) only at large width, while near the generalisation threshold it also depends on width.

### 4.5 Robustness: tasks, modulus, parameterisation, architecture

**Parameterisation (P7).** Fig. 5 and Table 4: with lr$\propto 1/w$ and lr·wd fixed, the $t_{\text{train}}$ slope for $w \geq 128$ is $-0.50$ $[-0.58, -0.42]$, the $t_{\text{val}}$ slope $-0.50$ $[-0.87, -0.09]$, and the $R$ slope $+0.005$ $[-0.37, +0.42]$: $R$ is flat $(420, 402, 423)$. With lr$\propto 1/w$ and wd fixed (decay rate $\propto 1/w$), $t_{\text{val}}$ *increases* with width (slope $+0.51$ $[+0.14, +0.92]$; 46k at $w$=256, 76k at $w$=512 with 2 seeds) and $R$ grows faster (slope $+1.03$ $[+0.57, +1.44]$). Hence "bigger models grok relatively later" is parameterisation-dependent; the absolute generalisation time follows the decay rate (compare $t_{\text{val}}$ at $w$=512: 16.3k fixed lr; 18.8k fixed decay rate; 76k decay rate $\propto 1/w$). **P7 is supported.** (The $w$=128 rows of the two scaled settings coincide because they are the same configuration run twice with identical seeds, which also confirms determinism.)

**Other operations and modulus (P5).** Table 2: $t_{\text{train}}$ falls with width in every task (P1 holds), and the absolute delay $D$ falls from $w$=128 to 256 for all five tasks. The relative delay behaves differently across tasks: $R$ rises from 128 to 256 for addition ($413 \to 492$) and subtraction ($342 \to 550$), is flat for multiplication ($433 \to 412$) and *falls* for $p$=97 ($343 \to 194$); for $a^2+b^2$, $R$ at $w$=128 is a lower bound ($\geq 909$, 2/4 censored), above $R$ at $w$=256 (627), so $R$ does not rise there either. **P5 is therefore only partly supported**: P1 and the decrease of $D$ generalise; the increase of $R$ does not. For $p$=97, $w$=64 is censored for all 3 seeds, i.e. the generalisation threshold moves up with $p$.

**Transformer (P6).** One-layer transformer, $d_{model} \in \{16, 32, 64, 128\}$ (5k–211k parameters), same optimiser, $T_{max} = 4 \times 10^4$ steps ($10^5$ at $d_{model}$=128 after the first runs did not finish), 4 seeds (3 at $d_{model}$=128). $t_{\text{train}}$ falls monotonically (1075, 345, 165, 100 steps; slope $-1.13$ $[-1.18, -1.08]$):

Table 3: Scorecard for registered predictions (details in the text).

| Prediction | Verdict | Evidence |
|---|---|---|
| P1 $t_{\text{train}} \downarrow$ in $w$ | supported | slope $-1.04$ to $-1.39$; all tasks |
| P2 $R \uparrow$, $D \not\downarrow$ | supported for add, $p$=53; not general | $R$ slope $+0.49$; fails $p$=97, mul |
| P3 label-agnostic fitting; threshold | partly: first half supported, second falsified at $f$=0.5 | $\rho$=0.9999; 16 vs 32 |
| P4 lr·wd scaling ($w$=128) | falsified | slope $-0.62$; ratio 1.64 |
| P5 other tasks/$p$ | partly | $D \downarrow$ everywhere; $R \uparrow$ only for add/sub |
| P6 transformer | partly: P1 and $R \uparrow$ supported; $D \downarrow$ not reproduced | slope $-1.13$; $D(128) \geq D(64)$; brittle 99% criterion |
| P7 parameterisation | supported | $R$ slope 0.0 vs $+1.0$ |
| P8 (post hoc) saturation | supported | lr×2: 1.8×; wd×3: 2.4× |

P1 holds. For $d_{model} \leq 32$ validation accuracy stays below 0.09 for all 8 runs within $4 \times 10^4$ steps; at $d_{model}$=64 three of four runs reach 99% (14–33k steps); at $d_{model}$=128 one of three reaches 99% (89k) and the other two plateau at 0.97–0.98 validation accuracy, so the 99% criterion is brittle for this model; with a 95% criterion (post hoc; Tab. 5) all three reach it (14.4k, 27.4k, 88k). The relative delay rises with width (median $R = 180$ at $d_{model}$=64 with one censored run; $\geq 890$ at 128 with 99%), but the absolute delay does not fall: median $D = 29.8$k at 64 versus a lower bound of 99.9k at 128 (99%; 2/3 censored), and under 95% the ranges overlap (8.6–31.6k vs 14.4–88k). So **P6 is partly supported** (P1 and the rise of $R$); the MLP behaviour of a falling $D$ is not reproduced, but our transformers are in the near-threshold regime ($d_{model} \leq 128$) where even MLPs show a steep $t_{\text{val}}(w)$, and the comparison has few seeds and a brittle criterion.

## 5 Discussion and limitations

**What the evidence supports.** In this controlled setting, width acts on grokking mainly by accelerating memorisation (a label-agnostic, lr-driven clock) much more than generalisation. Whether this makes grokking "more pronounced" depends on the definition: relative delay rises in the core setting, absolute delay falls, and a narrow-model regime exists where generalisation needs about $10^3\times$ the steps of memorisation ($R \approx$ 950–1300 at $w$=64, $f$=0.3, two runs) or does not happen within the budget. We did not find a width above which grokking disappears (largest $w$=512, 842k parameters; $R \approx 800$). The negative results are informative for experimental design: (i) with fixed lr, wd, wider models reach generalisation sooner in absolute time but delay is not independent of how the optimiser is scaled; (ii) a testbed that is "too small" will not show grokking within a typical budget; (iii) with more data some narrow models generalise directly.

**Relation to prior explanations.** The label-agnostic memorisation time and the importance of lr·wd at large width agree with weight-decay-driven pictures [Varma et al., 2023, Liu et al., 2023], and the parameterisation dependence is what one would expect if width acts through the lazy/rich balance [Kumar et al., 2024]; we did not measure laziness or circuits, so these remain interpretations, not tests.

**Limitations.** (1) One small modulus ($p$=53; one check at 97), one optimiser recipe (full-batch AdamW, $\beta_2$=0.98), and a single data fraction for most sweeps; minibatch noise, dropout and other regularisers were not studied. (2) $\leq 6$ seeds per cell, so CIs are wide, and 3 seeds in the probes; seed variation of $t_{\text{val}}$ is large (e.g. 26.9k–53.2k CI at $w$=128). (3) Censoring at $10^5$ steps (and $4 \times 10^5$ in one probe) means $t_{\text{val}}$ for narrow models is a bound, and the "threshold" depends on the budget. (4) $t_{\text{train}}$ has a 10-step resolution, which matters for $w \geq 256$ ($t_{\text{train}} \leq 40$). (5) The two-clock account was refined after seeing the pilot and main sweeps; the confirmatory value is in the pre-registered predictions with fresh seeds (P1–P7), while P8 and the three-regime picture are post hoc. (6) The transformer evidence is limited: $d_{model} \leq 128$, 3–4 seeds, a validation criterion that is brittle for this model (plateaus at 0.96–0.99), and only the near-threshold regime. (7) Width above 512 (MLP) and all depth effects were not studied; the MLP widths reach 842k parameters, near the $10^6$ budget. (8) No mechanistic analysis. (9) Results are for a 99% accuracy criterion.

## 6 Conclusion

On modular addition, model width speeds up memorisation roughly as $w^{-1}$ and generalisation more slowly, so grokking is relatively longer but absolutely shorter for wider MLPs at fixed lr and wd. This relative effect is not universal: it depends on the optimiser parameterisation, the modulus and the operation. Narrow models that can fit the data still need orders of magnitude more steps to generalise (or do not within a practical budget), and with more data some of them skip the delay. We hope the scorecard against our self-registered predictions, including falsified predictions, helps calibrate future explanations of delayed generalisation.

## References


Xander Davies, Lauro Langosco, and David Krueger. Unifying grokking and double descent. *arXiv preprint arXiv:2303.06173*, 2023.

Tanishq Kumar, Blake Bordelon, Samuel J. Gershman, and Cengiz Pehlevan. Grokking as the transition from lazy to rich training dynamics. In *International Conference on Learning Representations (ICLR)*, 2024.

Ziming Liu, Eric J. Michaud, and Max Tegmark. Omnigrok: Grokking beyond algorithmic data. In *International Conference on Learning Representations (ICLR)*, 2023.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

John X. Morris, Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M. Rush, Kamalika Chaudhuri, and Saeed Mahloujifar. How much do language models memorize? *arXiv preprint arXiv:2505.24832*, 2025.

Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. In *International Conference on Learning Representations (ICLR)*, 2020.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*, 2023.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

Kushal Tirumala, Aram H. Markosyan, Luke Zettlemoyer, and Armen Aghajanyan. Memorization without overfitting: Analyzing the training dynamics of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Vikrant Varma, Rohin Shah, Zachary Kenton, János Kramár, and Ramana Kumar. Explaining grokking through circuit efficiency. *arXiv preprint arXiv:2309.02390*, 2023.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.


## A Reproduction map and compute

### A.1 Experiments, scripts and artefacts

All code is in `code/`; `code/README.md` gives the commands. Each experiment is a function in `code/experiments.py`, run with `python sweep.py <name> <n_procs>`, which appends one JSON line per run (including the full accuracy/loss/parameter-norm curve) to `results/<name>.jsonl`. `python analysis.py` regenerates every figure (`paper/figs`), table (`paper/tables`) and number quoted in the text (`results/summary.json`).

| Experiment | Configuration (seeds) | Runs | Used for |
|---|---|---|---|
| `e1_mlp_width` | MLP, $p$=53, add, $f$=0.3, $w \in \{24, \ldots, 512\}$ (10 widths), $T_{max}$=$10^5$ (100–105) | 60 | Fig. 1, Tab. 1, Fig. 2c, 3 |
| `e2_random` | as e1, random train labels (100–102) | 30 | Fig. 2 |
| `e3_frac` | $f \in \{0.2, 0.5\}$, $w \in \{24..256\}$, real (100–102) and random labels (100–102), $T_{max}$=$6\times10^4$ | 96 | Fig. 2, 3, 6 |
| `e8_small` | $w \in \{8, 12, 16, 20\}$, $f \in \{0.2, 0.3, 0.5\}$, real and random labels (100–103), $T_{max}$=$6 \times 10^4$ (added after seeing that e2 fits from $w$=24) | 96 | Fig. 2, 3 |
| `e10_threshold` | $w \in \{72, 80, 88\}$, $f$=0.3 (100–105), $10^5$ steps (post hoc) | 18 | Sec. 4.3 |
| `e9_long` | $w \in \{48, 64\}$, $f$=0.3, $T_{max}$=$4\times10^5$ (100–102) (post hoc) | 6 | Fig. 3 |
| `e4_timescale` | $w$=128; 7 (lr,wd) pairs; $T_{max}$=$8\times10^4$ (100–103) | 28 | Fig. 4 |
| `e11_saturation` | $w$=384; (lr,wd)$\in$ $\{(2 \times 10^{-3}, 1), (10^{-3}, 3)\}$; $T_{max}$=$6\times10^4$ (100–103) (post hoc, P8) | 8 | Sec. 4.4 |
| `e6_generality` | sub, mul, $a^2+b^2$ ($p$=53) and add ($p$=97), $w \in \{64, 128, 256\}$ (100–103; 100–102 for $p$=97) | 45 | Tab. 2, Fig. 7 |
| `e7_param` | lr$\propto 1/w$, wd$\propto w$ or fixed; $w \in \{64..512\}$ (100–103; 100–101 at $w$=512) | 28 | Fig. 5, Tab. 4 |
| `e5_tf`, `e5b_tf_small`, `e12_tf128_long` | transformer, $d_{model} \in \{16, 32, 64, 128\}$, $p$=53, $T_{max}$=$4 \times 10^4$ (100–103); $d_{model}$=128 re-run with $T_{max}$=$10^5$ (100–102, post hoc) | 16 + 3 | Sec. 4.5, Fig. 8, Tab. 5 |
| `pilot`, `pilot_mlp` | exploratory pilot (seeds 0–2, 30k steps); both aborted when the compute budget was re-planned (partial) | 17 | motivates hypotheses only; not used in any reported statistic |

## A.2 Additional tables and figures

Table 4: Width-scaling variants (Fig. 5). The two scaled settings coincide at the reference width $w$=128 (same configuration and seeds).

| setting | width | lr | $t_{train}$ | $t_{val}$ | $R$ | cens./$n$ |
|---|---|---|---|---|---|---|
| fixed lr, wd | 64 | 0.001 | 210 | $\geq$100000 [100000,100000] | $\geq$476 | 6/6 |
| fixed lr, wd | 128 | 0.001 | 90 | 37200 [26900,53200] | 413 | 0/6 |
| fixed lr, wd | 256 | 0.001 | 40 | 19700 [16500,20700] | 492 | 0/6 |
| fixed lr, wd | 512 | 0.001 | 20 | 16300 [13200,19500] | 815 | 0/6 |
| lr$\propto 1/w$, lr·wd fixed | 64 | 0.002 | 115 | $\geq$100000 [100000,100000] | $\geq$871 | 4/4 |
| lr$\propto 1/w$, lr·wd fixed | 128 | 0.001 | 90 | 37800 [21400,63200] | 420 | 0/4 |
| lr$\propto 1/w$, lr·wd fixed | 256 | 0.0005 | 60 | 24100 [21000,25800] | 402 | 0/4 |
| lr$\propto 1/w$, lr·wd fixed | 512 | 0.00025 | 45 | 18800 [18800,18800] | 423 | 0/2 |
| lr$\propto 1/w$, wd fixed | 64 | 0.002 | 125 | $\geq$100000 [92800,100000] | $\geq$771 | 3/4 |
| lr$\propto 1/w$, wd fixed | 128 | 0.001 | 90 | 37800 [21400,63200] | 420 | 0/4 |
| lr$\propto 1/w$, wd fixed | 256 | 0.0005 | 60 | 46200 [34400,61600] | 770 | 0/4 |
| lr$\propto 1/w$, wd fixed | 512 | 0.00025 | 45 | 76300 [65200,87400] | 1744 | 0/2 |

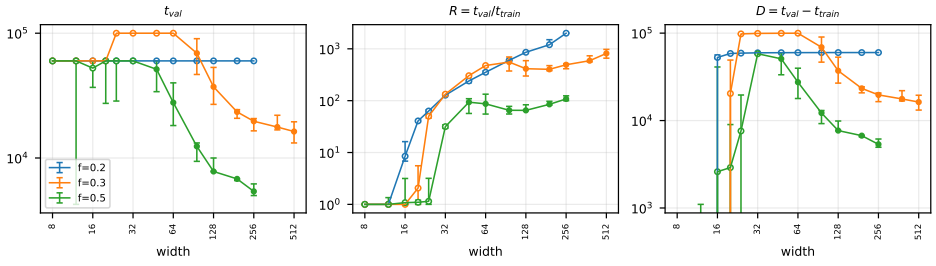



Figure 6: $t_{val}$, $R$, $D$ vs. width for three training fractions (MLP, $p$=53; $10^5$ steps for $f$=0.3, $6\times10^4$ for $f$=0.2, 0.5; hollow: mostly censored). Only the $f$=0.3 line extends beyond $w$=256.

## A.3 Environment, seeds, cost

Python 3.11.15, PyTorch 2.15.0+cpu (CPU only, 1 thread per run), NumPy 2.4.6, SciPy 1.17.1, Matplotlib 3.11.2; LaTeX: TeX Live `pdflatex/latexmk`. Seeds: 0–2 (pilot), 100+ (all reported experiments); the seed sets both model initialisation and the train/validation split (split seed = 10000+seed). Runs are deterministic for a given (config, seed) on a given machine (verified: duplicate

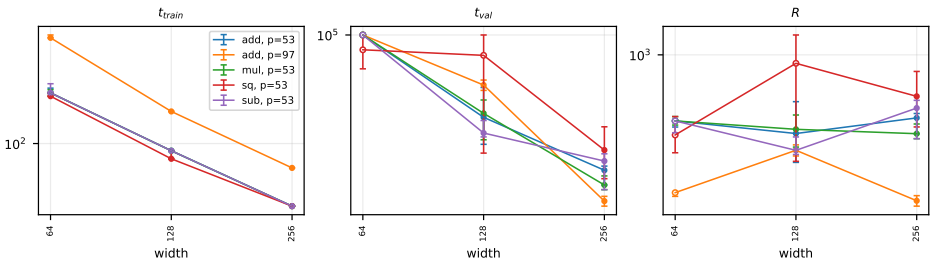



Figure 7: Other operations and modulus (Tab. 2).

Table 5: Transformer runs (seeds 100–103; 100–102 at $d_{model}$=128). Per-seed times to 99% and (post hoc) 95% validation accuracy; −: not reached within the budget. Max. val. acc.: highest validation accuracy over the run.

| $d_{model}$ | params | budget | $t_{train}$ | $t_{val}$, 99% (per seed) | $t_{val}$, 95% (per seed) | max val. acc. |
|---|---|---|---|---|---|---|
| 16 | 5k | 40k | 1075 | −, −, −, − | −, −, −, − | 0.04, 0.02, 0.08, 0.04 |
| 32 | 16k | 40k | 345 | −, −, −, − | −, −, −, − | 0.02, 0.07, 0.03, 0.02 |
| 64 | 57k | 40k | 165 | −, 26.2k, 14.2k, 33.4k | −, 19.8k, 8.6k, 31.6k | 0.03, 0.99, 0.99, 0.99 |
| 128 | 211k | 100k | 100 | −, 89.0k, − | 27.4k, 88.0k, 14.4k | 0.99, 0.99, 0.99 |

configurations in e7 gave identical results). The sandbox provided about 4–5 effective CPU cores shared with other tenants; the per-run wall-clock seconds in `results/*.jsonl` include contention. Summed per-run wall-clock time over all stored runs is given by `python code/compute_stats.py` (`code/logs/compute_stats.log`); it is 78 hours over all stored runs (434 reported runs plus 12 duplicated transformer runs and 17 pilot runs), most of it contention-inflated; the whole project took roughly 11 hours of wall-clock on this shared slice.

### A.4 Deviations, failed or abandoned runs, and honest notes

- **Template.** The official NeurIPS 2025 style file (the newest we could obtain: the 2026 archive was not available at the expected URL) is used in `preprint` mode (non-anonymous author line), with the NeurIPS checklist included.
- **Compute-driven choices.** $p$=53 rather than 97 (the latter is $\approx 3\times$ costlier; checked only for 3 widths); transformer limited to $d_{model} \leq 128$ and $4 \times 10^4$ steps. An exploratory transformer pilot with $d_{model}$ up to 256 and an MLP pilot with $w \in \{512, 1024\}$ were stopped for cost; their partial output is in `results/pilot*.jsonl` and is not used.
- **Registration timing.** `PREREG.md` was written before e1 and e2 were launched. It is a self-registered file in the workspace (not an external registry), and the results of e2 (including the near-identical memorisation times) were visible before e3–e7 were launched; no prediction was changed afterwards, but P3's second half and the choice of e3/e4 grids could in principle have been influenced by e2.
- **Post-hoc additions** (marked in the text): e8, e9, e10, e11, the three-regime interpretation, the saturation reading, and the choice to report $p$=97 for only 3 widths. The registered predictions P1–P7 were not edited after the sweeps (the file `code/PREREG.md` has an explicit addendum for P8).
- **Analysis changes after seeing data.** The slope for $t_{\text{train}}$ and $t_{\text{val}}$ is reported over fully-fitting/uncensored widths because including censored widths mixes bounds with values; the all-width slope for $t_{\text{train}}$ is also given. The e4 regression treats censored runs as bounds (biased toward flatter slopes), a censoring-aware estimator (Tobit or survival regression) was not implemented. For the transformer, the 95% validation criterion was added after observing plateaus just below 99%.
- **Parallel duplicates.** Because of a restart, some transformer configurations were queued twice; duplicate keys are dropped when loading (first result kept).
- **Literature.** References were checked to exist via their arXiv abstract pages (`code/check_refs.py`, `code/logs/refs_check.log`); statements about their content

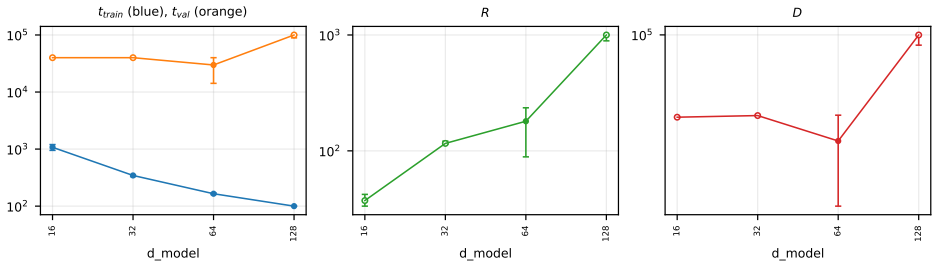



Figure 8: Transformer: $t_{train}$ and $t_{val}$ (99%), $R$, $D$ vs $d_{model}$ (medians, bootstrap CIs; hollow: mostly censored).

in Sec. 2 follow the one-line descriptions in the task's starter list and our prior knowledge; the full texts were not re-read during this run. No other literature search was run.

- **LLM usage.** The research design, code, experiments, analysis and this manuscript were produced by an autonomous LLM-based agent running unattended; no human reviewed the intermediate results. The studied models are small MLPs and transformers trained from scratch; no LLM is part of the experimental pipeline.
- **Restricted content.** No search for, access to, or reading of the restricted paper(s) described in the task brief was performed; no web pages other than the NeurIPS style-file archive, pip packages and the arXiv abstract pages listed above were accessed. Nothing restricted was encountered.

## NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Sec. 1 state the measured relations (with CIs) and the falsified predictions; scope limits ($p$=53, one optimiser recipe, $\leq$ 6 seeds) are stated in the abstract and Sec. 5.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Sec. 5 (Discussion and limitations) lists nine limitations (modulus, recipe, seeds, censoring, resolution, post-hoc analyses, transformer evidence, scale range, no mechanistic analysis).

   Guidelines:

   - The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: The paper contains no theoretical results.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sec. 3 and App. A give data generation, architectures, optimiser, metrics, seeds and the statistics; code regenerates every number.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: All code is in the supplementary `code/` directory with a README listing environment versions, exact commands and run times; data are synthetic and generated in memory.

   Guidelines:

   - The answer NA means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: Sec. 3 and App. A: architectures, initialisation, optimiser settings, evaluation schedule, splits and seeds; hyperparameters were fixed a priori (they follow common grokking setups) and varied only in the reported sweeps.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Medians with 95% bootstrap CIs over seeds (seed = initialisation and data split) in all figures/tables; log-log slopes and Spearman correlations have bootstrap CIs; censoring is handled as lower bounds. $n \leq 6$ so CIs are coarse; no multiple-testing correction (stated in Sec. 4).

   Guidelines:

   - The answer NA means that the paper does not include experiments.

- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: App. A.3: CPU only (about 4–5 effective shared cores), per-run wall-clock recorded in the results files, about 65 summed run-hours; pilot and aborted runs are disclosed.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses only synthetic data, no human subjects, and follows the Code of Ethics as far as we can assess.

   Guidelines:

   - The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [NA]

Justification: Foundational study of small synthetic tasks; we see no direct societal impact beyond general ML understanding.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: [NA]

    Justification: No models or data with misuse risk are released.

    Guidelines:

    - The answer NA means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: Only open-source libraries (PyTorch, NumPy, SciPy, Matplotlib) and the NeurIPS style files are used, cited in App. A.3; no third-party data.

    Guidelines:

    - The answer NA means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The new code and result files are documented in `code/README.md`; no new datasets (data are generated by a few lines of code).

    Guidelines:

    - The answer NA means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: No crowdsourcing or human subjects.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

    Justification: No human subjects.

    Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: The research was conceived, run, analysed and written by an autonomous LLM-based research agent (no human in the loop during the run); this is declared in App. A.4. LLMs are not part of the studied models.

    Guidelines:

    - The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy (`https://neurips.cc/Conferences/2025/LLM`) for what should or should not be described.